



# AI-Driven Intrusion Detection Systems (IDS) on the ROAD Dataset: A Comparative Analysis for Automotive Controller Area Network (CAN)

Lorenzo Guerra  
LTCl, Télécom Paris, Institut  
Polytechnique de Paris  
Palaiseau, France  
lorenzo.guerra@telecom-paris.fr  
Ampere Software Technology  
Guyancourt, France  
lorenzo.guerra@ampere.cars

Thomas Chapuis  
Ampere Software Technology  
Guyancourt, France  
thomas.chapuis@ampere.cars

Linhan Xu  
LTCl, Télécom Paris, Institut  
Polytechnique de Paris  
Palaiseau, France  
linhan.xu@utt.fr

Guillaume Duc  
LTCl, Télécom Paris, Institut  
Polytechnique de Paris  
Palaiseau, France  
guillaume.duc@telecom-paris.fr

Van-Tam Nguyen  
LTCl, Télécom Paris, Institut  
Polytechnique de Paris  
Palaiseau, France  
van-tam.nguyen@telecom-paris.fr

Paolo Bellavista  
University of Bologna  
Bologna, Italy  
paolo.bellavista@unibo.it

Pavlo Mozharovskyi  
LTCl, Télécom Paris, Institut  
Polytechnique de Paris  
Palaiseau, France  
pavlo.mozharovskyi@telecom-paris.fr

## Abstract

The integration of digital devices in modern vehicles has revolutionized automotive technology, enhancing safety and the overall driving experience. The Controller Area Network (CAN) bus is a central system for managing in-vehicle communication between the electronic control units (ECUs). However, the CAN protocol poses security challenges due to inherent vulnerabilities, lacking encryption and authentication, which, combined with an expanding attack surface, necessitates robust security measures. In response to this challenge, numerous Intrusion Detection Systems (IDS) have been developed and deployed. Nonetheless, an open, comprehensive, and realistic dataset to test the effectiveness of such IDSs remains absent in the existing literature. This paper addresses this gap by considering the latest ROAD dataset, containing stealthy and sophisticated injections. The methodology involves dataset labeling and the implementation of both state-of-the-art deep learning models and traditional machine learning models to show the discrepancy in performance between the datasets most commonly used in the literature and the ROAD dataset, a more realistic alternative.

## CCS Concepts

• **Security and privacy** → **Intrusion detection systems**; *Network security*; • **Computing methodologies** → **Machine learning**.

## Keywords

AIoT, Intrusion Detection System, IDS, Controller Area Network, CAN, ROAD Dataset

## ACM Reference Format:

Lorenzo Guerra, Linhan Xu, Paolo Bellavista, Thomas Chapuis, Guillaume Duc, Pavlo Mozharovskyi, and Van-Tam Nguyen. 2024. AI-Driven Intrusion Detection Systems (IDS) on the ROAD Dataset: A Comparative Analysis for Automotive Controller Area Network (CAN). In *Proceedings of the 2024 Cyber Security in CarS Workshop (CSCS '24)*, October 14–18, 2024, Salt Lake City, UT, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3689936.3694696>

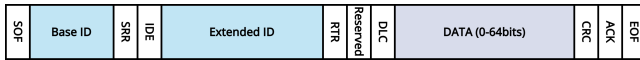
## 1 Introduction

With the rapid development of Internet of Things (IoT) and Internet of Vehicles (IoV) technologies, network-managed vehicles, such as Autonomous Vehicles and Connected Vehicles, are progressively taking the place of traditional automobiles. Within IoV systems, there are typically two core components: intra-vehicle networks (IVNs) and external networks. In IVNs, the core infrastructure is the Controller Area Network (CAN) bus, which enables communication among Electronic Control Units (ECUs). In contrast, external vehicular networks utilize Vehicle-to-Everything technology to establish links between intelligent vehicles and other IoV elements.



This work is licensed under a Creative Commons Attribution International 4.0 License.

CSCS '24, October 14–18, 2024, Salt Lake City, UT, USA  
© 2024 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-1232-6/24/10  
<https://doi.org/10.1145/3689936.3694696>



**Figure 1: CAN bus data frame.**

The surge in connectivity has created a breeding ground for vulnerabilities. Malicious actions can be injected remotely through wireless communication systems, or physically through vectors like OBD-II ports, USB interfaces, CD drives, and more. In response to the growing threats, Intrusion Detection Systems (IDS) have emerged as a crucial defense mechanism. Leveraging the outstanding performance demonstrated by artificial intelligence, we aim to showcase the capabilities of various deep learning and conventional machine learning techniques in identifying anomalous behaviors.

By taking into consideration that the effectiveness and robustness of an Intrusion Detection System (IDS) are not solely dependent on the capabilities of the AI model in use, but also on the characteristics of the training dataset, we initiate the process of training different models using a unique dataset [28] that includes 11 discrete categories of stealthy attacks, validated on a real vehicle, marking a significant departure from previous datasets, which were predominantly composed of few unrealistic attacks [14]. Starting by labeling [28], this paper’s contributions extend to the evaluation of a wide range of supervised deep learning techniques, including the Transformer-based Attention Network (TAN) developed by Nguyen et al. (2023) [21], the Deep Convolutional Neural Network (DCNN) developed by Song et al. (2020) [24], and the model based on Long Short-Term Memory (LSTM) developed by Hossain et al. (2020) [11], alongside various machine learning methods such as LightGBM, Random Forest (RF) and the LCCDE model [29] applied to the aforementioned dataset. In conclusion, we conduct a comparative analysis, employing additional established datasets such as the HCRL Car Hacking dataset and the In-Vehicle Network Intrusion Detection Challenge dataset, eventually suggesting improvements.

The remainder of this paper is organized as follows. In Section 2, we give a general background on the CAN protocol, its vulnerabilities and we review the literature on the topic. Section 3 provides an overview of the datasets used, with a specific focus on ROAD and its attacks. Subsequently, in Section 4, we offer a concise depiction of the employed IDS techniques, followed by the presentation and analysis of the experimental setup and results in Section 5. To conclude, Section 6 provides a summary of our findings.

## 2 Background and Literature Review

### 2.1 Controller Area Network Bus

**2.1.1 Protocol.** The CAN Bus, developed by Bosch in the 1980s for Local Area Networks (LAN), is a message-based protocol designed to enable numerous electric components, e.g., microcontrollers, ECUs, sensors, devices, and actuators, throughout the in-vehicle system to communicate with each other through a dual-wire bus. A standard CAN data frame (or message), featured in Figure 1, consists of several fields. The most important fields are described as follows:

- **Identifier (CAN ID):** The CAN ID is represented by 11 bits in the standard format (CAN 2.0A) or 29 bits if the extended

format is used (CAN 2.0B). It determines the priority of each message within the network. The CAN physical layer uses wired-AND signalling, meaning that a dominant level would be represented by a logical 0 and a recessive level by a logical 1, so the lower the ID, the highest the priority.

- **Data Length Code (DLC):** Located within the control field, this parameter indicates the number of bytes present in the data field, with a valid range extending from 0 to 8 bytes.
- **Data field:** Contains the actual contents of the message, which can be up to 8 bytes in size.
- **Cyclic Redundancy Check (CRC):** a built-in error detection mechanism that ensures the reliability of the communication by detecting transmission errors and requesting a re-transmission if necessary.

**2.1.2 CAN vulnerabilities.** Because of its simplicity, the CAN bus lacks essential security measures:

- **Missing Authentication:** ECUs do not have to authenticate to be able to transmit messages on the bus, every device can connect and it is immediately able to receive and transmit data, which makes the bus very flexible but vulnerable to intruders. In a normal scenario, the ECU corresponding to a precise functionality, for example, the wheel speed sensor communicating the speed at which the vehicle is going, will only send information concerning its domain, and it would never transmit frames reporting the coolant temperature. But protocol-wise nothing prevents the ECU from sending any type of information, which in case of an intruder could lead to spoofing attacks performed with ease.
- **Missing Encryption:** the data transmitted on the CAN bus is transparent and it is broadcasted to every ECU, meaning that any attacker could eavesdrop on unencrypted communication without anyone noticing. Combined with the missing authentication, this can lead to more complex attacks harder to detect since the normal behavior of ECUs can be anticipated, potentially suppressing the real messages. Spoofing attacks are also easy to perform by exploiting the transparency and the missing authentication, making detection exceptionally hard if such attacks are performed correctly.

Some approaches to secure the communication on the bus have been experimented during the years [10], but unfortunately they have not gained enough traction. Consequently, the protocol does not respect 2 of the 3 fundamental properties focused on by information security principles, which are summarized by the CIA triad, *Confidentiality, Integrity, Availability*:

- **Confidentiality** is not respected because of the missing encryption, since any intruder is able to eavesdrop on all the traffic just by connecting to the CAN bus.
- **Integrity** is respected thanks to the CRC, detecting bit errors, while error frames force the bus dominant longer than the bit stuffing rule. Furthermore, automatic re-transmission occurs until the error limit is reached.
- **Availability** is not respected because the authentication is missing, anyone connected to the bus can flood it with high-priority frames, overwhelming the communication and preventing the ECUs from sending any message.

The most difficult task for an attacker, considering the critical vulnerabilities of the CAN protocol, would be to access the bus. The main attack vectors are two [12], one is physical access, the other one is through one of the many wireless interfaces.

To reach the bus through physical access, the simplest way would be through the OBD-II port, mentioned in the previous section. This connector exposes the CAN bus without needing any authentication, so an intruder would be able to eavesdrop and inject messages just by connecting a malicious component to it. It is also possible to leave the component attached indefinitely, concealing it in the dashboard, where the OBD-II port is most commonly found. Another way to get physical access would be to install a counterfeit component in the car, so the supply chain itself needs to be secured, and only authorized repair shops should be able to replace components. Reaching the CAN bus through one of the wireless interfaces is less simple and would require such interfaces to be themselves vulnerable. Because of the large number of these devices in recent vehicles, it is not rare to find a vulnerability in one of them leading to access to the CAN bus.

A challenge-response mechanism protects ECUs from being re-flashed and reading sensible memory. This challenge-response is performed through a seed-to-key challenge, but both seed and key are fixed and stored in the device. The system allows only an attempt every 10 seconds, but due to the low complexity, it is sufficient to have access for 7 and a half days to be able to crack the challenge-response and therefore reflash every ECU [12].

The CAN bus exposes also a diagnostic system called Device Control, a powerful tool that is meant to be used for testing purposes, and it can modify the internal state of the ECUs. Paired with the vulnerabilities already described, it is easily exploitable and it can lead to dangerous consequences. This service takes an argument called a Control Packet Identifier (CPID), which specifies a group of controls to override, and each one can take up to five bytes as parameters, specifying which controls in the group are being overridden, and how to override them [12].

## 2.2 Literature Review and Research Gaps

One of the first security analyses on real cars has been made in 2010 by Koscher et al. [12], assessing the behavior of the vehicle during specific attacks. They leveraged the security vulnerabilities of CAN to control a wide range of critical functions, like stopping the engine, disabling the brakes, or even braking on-demand individual wheels. However, they do not yet prove that these attacks are feasible remotely. In 2011 Checkoway et al. [5] analyzed purely the external attack surface of one vehicle to assess if it might be vulnerable to remote attacks exploiting the wireless interfaces present in the car, and they found out that it is indeed possible through multiple short-range and long-range vectors like Bluetooth or cellular interfaces. The results of Checkoway et al. are particularly concerning if we take into consideration the paper published by Koscher et al.: these two studies show that intruders can preserve their anonymity controlling critical features of vehicles from long distances, potentially causing disastrous, life-threatening consequences.

Charlie Miller and Chris Valasek are two researchers working on automotive cybersecurity who gained a reputation during the years by publishing multiple relevant papers on the topic. In 2013

[26] they tinkered with the CAN bus, exploiting its vulnerabilities to take control of critical functionalities of two different cars; in 2014 [17] they took Checkoway et al.'s previous research further by identifying the remote attack surface for a large range of vehicles, identifying the ECUs' capabilities and estimating the difficulty of remote exploitation; in 2015 [18] they brought to conclusion 3 years of research disclosing a long-range remote attack that can be performed on Jeep Cherokees without any physical interaction, causing Fiat Chrysler to recall 1.4 million vehicles [20]. One year later, in 2016 [19], they published another paper explaining how they performed a masquerade attack on the Jeep Cherokee and how they were able to deal with message confliction.

Charlie Miller and Chris Valasek believe that attack detection represents an inexpensive and accurate way to greatly improve the security of CAN networks [17]. Current research on the topic focuses extensively on machine learning [1] and deep learning [15] approaches in order to reach an increasingly higher accuracy. Unfortunately, these systems are not yet perfect: in 2021 Bhatia et al. [2] explained how they developed a masquerade attack called DUET which could evade detection of Voltage-based Intrusion Detection Systems. This type of IDS is able to observe the voltage fingerprint of ECUs, thus recognizing the ones belonging to attackers. However, the DUET attack showed that VIDS are not a silver bullet against intrusions, so the authors introduced a lightweight defense mechanism called RAID, suggesting to modify the frame format in a protocol-compatible way in order to safeguard against the corruption of ECUs' voltage fingerprints.

According to its design, an Intrusion Detection System can be categorized into two major families: anomaly-based and signature-based detection methods [16]. While signature-based IDSs compare the traffic with a database of known attack patterns, anomaly-based IDSs can be powered by machine learning techniques to model the normal behavior of the machine and they are able to detect unseen attacks as well.

Although prior AI-based IDS models have yielded to promising results [29][11][24][21], there exists a remarkable gap in testing them on a more realistic dataset including sophisticated attack patterns. It is worth noting that the effectiveness of an IDS model depends heavily on the quality of the data it was trained on. Previously introduced approaches were tested on datasets characterized by a limited number of attack types, typically only three or four, which are considered simple and unrealistic as they are conducted in a testbed environment [14]. These attacks are easily detectable and not necessarily verified on a real car. To build a robust IDS able to detect stealthy attacks it is imperative to test the model against high fidelity attack scenarios.

## 3 Datasets Description

### 3.1 ROAD Dataset and Labeling

The Real ORNL Automotive Dynamometer (ROAD) dataset [28] includes 33 attack captures spanning 30 minutes and 12 ambient captures over 3 hours (Table 1). It originates from a real undisclosed vehicle driven on a dynamometer of the Oak Ridge National Laboratory (ORNL) during 3 and a half hours of activity, reproducing various benign driving behaviors, while ambient and attack data were collected.

The logs are collected with the candump utility [3] and the output format is the following:

$\underbrace{(1040000000.000682)}_{UnixTimestamp}$ 
 $\underbrace{can0}_{Channel}$ 
 $\underbrace{0BA}_{ID}$ 
 $\underbrace{\#04B7EC04000602C8}_{DataField}$

**Table 1: Overview of the attacks contained in the ROAD dataset.**

| Attack Types                               | # Attack Samples |
|--------------------------------------------|------------------|
| Accelerator Attack in Drive                | -                |
| Accelerator Attack in Reverse              | -                |
| Correlated Signal Fabrication Attack       | 5493             |
| Correlated Signal Masquerade Attack        | 5493             |
| Fuzzing Attack                             | 1059             |
| Max Engine Coolant Temp Fabrication Attack | 43               |
| Max Engine Coolant Temp Masquerade Attack  | 43               |
| Max Speedometer Fabrication Attack         | 11694            |
| Max Speedometer Masquerade Attack          | 11694            |
| Reverse Light Off Fabrication Attack       | 5480             |
| Reverse Light Off Masquerade Attack        | 5480             |
| Reverse Light On Fabrication Attack        | 8034             |
| Reverse Light On Masquerade Attack         | 8034             |

The dataset features eleven distinct classes depicted in Table 1 for which metadata for labeling was provided. An extra attack called "accelerator attack" is included in the dataset, although the labeling information was not disclosed by the authors due to the seriousness of the vulnerability.

The attack types can be summarized in 3 categories:

- (1) *Fuzzing Attack*: Frames with cycling IDs have been injected with a payload set to the maximum value (0xFFFFFFFF). This attack caused various physical effects, for example: the accelerator pedal became ineffective, the dash lights and head-lights were illuminated and the seat positions moved.
- (2) *Targeted ID Fabrication Attacks*: These attacks are performed injecting a message immediately after the legitimate message targeting a specific ID. In this case only the target signal is modified. Differently from most of public datasets, the fabrication attacks in the ROAD dataset contain one single frame injected between ambient frames of the same ID, since only a single injected message is needed after each legitimate message for this attack to be successful. Because these are real, physically verified attacks with the minimally occurring injected frames (due to the flam delivery), they provide perhaps the best (i.e., most stealthy/most difficult to detect), current, public data for testing frequency-based IDSs [28].
- (3) *Masquerade Attacks*: These attacks are performed during post-processing by removing the legitimate target ID frames preceding each injected frame, making it appear as though only the spoofed messages are present during the injection interval. The paper explains that even though the effect on the vehicle was verified and the CAN protocol was not violated, this attack has not been performed directly on the

vehicle because of the complexity of the implementation. There are no available public captures of real masquerade attacks. This attack can be defined as *Timing Opaque (T.O.)* because it does not disrupt normal timing or ID distributions, so it is very unlikely that frequency-based approaches could provide accurate detection, while the previous types can be defined as *Timing Transparent (T.T)* because a frequency-based approach would be able to detect them.

The log files themselves were not labeled, but the authors provided a file with the capture metadata in JSON format that allowed us to assign the labels based on the conditions specified (i.e. the interval during which the attack occurred, the ID used during the injection and the injected bytes in the data field).

### 3.2 HCRL Car-Hacking Dataset

The Car-Hacking dataset [7], according to Rajapaksha et al. (2023) [22], is the most popular dataset in literature, and in many recent publications it seems to be used without citation [28]. It is generated by injecting CAN messages into the Hyundai YF Sonata's CAN bus through the OBD-II port and includes both attack and regular messages. Each sample is composed by a timestamp, the CAN ID, the DLC and up to 8 bytes of the data field. The attack types are four:

- *DoS attack*: This attack is simply performed by transmitting messages on the CAN bus with the ID containing only dominant bits (0) (i.e. ID=0x000) every 0.3 milliseconds. As seen in the previous section, the CAN bus is using wired-AND signaling, which means that the messages with the ID containing the largest number of dominant bits will be prioritized over any other ECU, which will have to patiently wait for the attack frame to be completely transmitted before trying to send another one. Therefore, an attacker with access to the CAN bus would simply need to continuously inject messages using this technique and the correct functioning of the system would be disrupted, needing a manual restart of the car [12].
- *Fuzzy attack*: CAN messages with spoofed random ID and DATA values are injected every 0.5 milliseconds, causing an unexpected behavior of the vehicle.
- *RPM/Gear attack*: CAN messages with spoofed ID relative to the RPM or Gear ECU are injected every 1 millisecond.

For each of the four attack datasets, the total number of messages is approximately 4,000,000 (Table 2), of which 13% to 16% are attack messages, therefore maintaining a relatively balanced attack proportion. There is a fifth file containing normal data in log format, but unfortunately, most of the papers employing this dataset seem to exclude the data contained in this additional log file, likely due to its limited benefits, as the other four CSV files already provide a considerable amount of benign messages.

### 3.3 In-Vehicle Network Intrusion Detection Challenge Dataset

The In-Vehicle Network Intrusion Detection Challenge dataset [9] was used during the "Information Security R&D Data Challenge 2019" and includes traffic data collected from three vehicles in

**Table 2: Overview of the Car-Hacking Dataset.**

| Name of the Attack   | # Total Messages | # Normal Messages | # Attack Messages |
|----------------------|------------------|-------------------|-------------------|
| DoS Attack           | 3,665,771        | 3,047,062         | 587,521           |
| Fuzzing Attack       | 3,838,860        | 3,259,177         | 491,847           |
| Gear Spoofing Attack | 4,443,142        | 3,805,725         | 597,252           |
| RPM Spoofing Attack  | 4,621,702        | 3,925,329         | 654,897           |
| Normal Run Data      | 988,871          | 988,871           | —                 |

stationary state: a Hyundai Sonata, a Kia Soul, and a Chevrolet Spark. We have used the datasets collected during the preliminary round, since they include three attack categories (Flooding, Fuzzy, and Malfunction) compared to the two attack categories of the final round. The attack types are the same of the Car-Hacking dataset, but with different names. The dataset is labeled and, like the Car-Hacking dataset, each sample is represented by a timestamp, a CAN ID, the DLC and the data field. For all three types of vehicles, the proportion of "Flooding" attacks is relatively high, about 20% of the total messages, compared to only about 8% for the other two types (Table 3). The application of the same attack types and setup on three different vehicles makes this dataset unique and enables us to test transfer learning approaches on it, in order to improve the generalizability of the model.

## 4 Fundamentals of Machine Learning and Deep Learning IDS Models

### 4.1 LCCDE and ML Approaches

To compare neural network approaches to traditional machine learning ones, gradient boosting and decision tree based approaches have also been tested on the available datasets. Specifically we considered Random Forest, LightGBM and the LCCDE model. These methods, like other machine learning algorithms, are preferable because of their speed, efficiency and explainability. Furthermore, the Leader Class and Confidence Decision Ensemble (LCCDE) [29], an ensemble model based on gradient boosting algorithms, was able to reach state-of-the-art results. The ensemble algorithm is divided into two stages: training and prediction. In the training stage, the three base models are trained on the labeled data. For each class, the leader is selected based on the best F1-score of the models, obtained from cross-validation. In case of a tie in F1-score, the model with the highest speed is chosen.

In the prediction step, every model is first used to make predictions. Decisions are then taken based on the predicted classes:

- If the predicted classes are the same, then this class will be the final prediction.
- If the predicted classes are all different, the leader model associated with each predicted class is cross-referenced with the base learner that made the corresponding prediction. If a single pair of leader and base models align, their jointly predicted class is adopted as the final prediction. In cases where multiple pairs exist, the predicted class with the highest confidence level is selected as the ultimate prediction.

- If only two of the predicted classes match, then the respective leader model is chosen for the final prediction.

The traditional machine learning models and the LCCDE were fed single frames with ID, DLC (if available) and data field as input and all these approaches involve multiclass classification, allowing a more precise prevention system based on the specific attack type detected.

### 4.2 LSTM-based Model

LSTMs are a type of Recurrent Neural Networks (RNNs) that are frequently and effectively applied for sequence classification tasks, therefore we have selected the state-of-the-art LSTM model described by Hossain et al. [11] to test this approach on the previously specified datasets, since it achieved outstanding results on the Survival Analysis Dataset [8]. The authors have tested many hyperparameters for both binary and multiclass classification. The best performing model was composed by a single LSTM layer with 512 units using single time steps and batches of size 512 as input. Each input sample includes ID, DLC (if available) and the bytes of the data field. We have performed multiclass classification on every dataset and binary classification only on the ROAD dataset, to compare its results with the DCNN and the TAN models. However, the approach of analyzing single messages can be considered vulnerable to replay attacks and since the authors used single time steps, the utility of a LSTM for this purpose is questionable.

### 4.3 DCNN

The IDS model proposed by Song et al. [24] is based on a convolutional neural network (CNN), aimed at detecting the patterns of IDs appearing in the CAN traffic. During the training phase, the model employs a frame builder to extract sequences of IDs from the logged traffic data. To facilitate CNN processing, which typically works with grid-formatted input, the temporal sequential patterns of CAN traffic are processed as spatially-local correlation. The frame builder extracts the 29-bit identifiers, which are the maximum number of bits for representing an ID when using the 'extended format' [23], from the most recent 29 messages and builds a  $29 \times 29$  bitwise frame by stacking them. A sliding window of size 29, with a step of 29 is used. This means that attack patterns might extend across 2 different frames and could not be easily recognized by the DCNN, which is unable to retain memory across frames. For this reason, we tested a different approach modifying the step of the sliding window to one, enabling the model to catch every pattern involving IDs and extending for any 29 consecutive frames. The frame creation process is illustrated in Figure 2.

The frame builder uses pure bits as input data without preprocessing, ensuring computational efficiency, given the high volume of messages on a CAN bus. The bit representation of the identifier explicitly illustrates the fluctuation of identifier patterns. The bit representation of the identifier, denoted as:

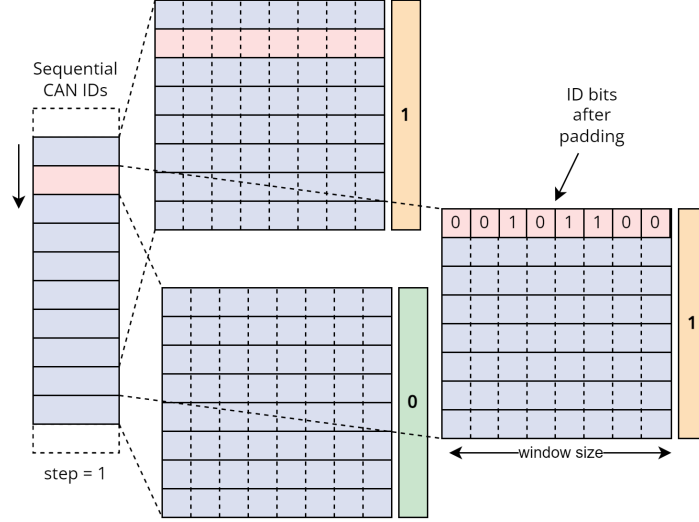
$$ID = b_i \text{ (for } i = 0, \dots, 28) \quad (1)$$

is assembled into a data frame as:

$$FRAME = b_{ij} \text{ (for } i = 0, \dots, 28) \quad (2)$$

**Table 3: Overview of In-Vehicle Network Intrusion Detection Challenge dataset.**

| # Name of the Vehicle | # Total Messages | # Normal Messages | # Flooding Messages | # Fuzzy Messages | # Malfunction Messages |
|-----------------------|------------------|-------------------|---------------------|------------------|------------------------|
| Chevrolet Spark       | 313,934          | 291,897           | 14,999              | 3,043            | 3,995                  |
| Hyundai Sonata        | 388,173          | 353,783           | 17,093              | 9,095            | 8,202                  |
| Kia Soul              | 569,516          | 527,061           | 16,072              | 21,613           | 4,770                  |

**Figure 2: Frame creation process of the DCNNv2.**

where  $b_{ij}$  represents the  $j$ -th bit value of the  $i$ -th ID in a frame. Each frame (grid) is then labeled as one if it contains an attack message, or zero for normal traffic.

The DCNN model itself is based on the Inception-ResNet model [25], which is originally designed to classify  $299 \times 299 \times 3$  input image data into 1000 classes, but has been reduced by the authors to classify more efficiently  $29 \times 29 \times 1$  into only 2 classes. Compared to the original model the proposed DCNN excludes the Inception-ResNet-C block, while keeping the stem, Inception-ResNet-A, -B, Reduction-A and -B, average pooling, dropout, and softmax blocks, although they have been internally simplified to adapt to the studied case. The final model uses only 2% of the memory of the original one, and only the 18% of the original parameters, making it more suitable for use in restricted environments, like automotive applications.

#### 4.4 TAN

The last approach for intrusion detection that we adopted in this study relies on a transformer-based architecture developed by Nguyen et al. [21]. Differently from the original transformer architecture [27], the TAN architecture only uses the encoder, the most essential component for classification, thus resembling the highly influential ViT model produced by Google [6]. Furthermore, the model has been tested for transfer learning, which aims to address a significant challenge in in-vehicle intrusion detection. This method enables the application of a model trained on a large amount of

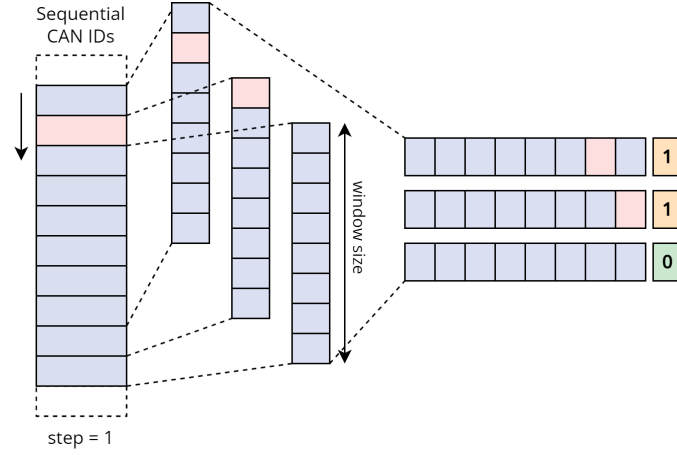
data from a specific vehicle to a different vehicle for which less data is available.

The authors have explored two different approaches, one which analyses single messages and their every field, but it is vulnerable to replay attacks, and one that analyzes sequences of CAN IDs. We have chosen the latter as it is a more solid choice and it could achieve very good results, despite part of the information relevant to intrusion detection is discarded. The sequence creation process involves a sliding window with a step equal to one. Multiple window sizes have been tested and we have selected a window size of 16 because of its balance between performance and memory requirements. The sequence creation process is illustrated in Figure 3 and consists in grouping sequences of consecutive CAN IDs, which are labeled as 1 if they contain an attack message, or 0 otherwise. This choice implies the approach of binary classification, therefore impeding us to distinguish what types of attack are detected.

## 5 Experimental Environment and Results Analysis

### 5.1 Comprehensive Framework: Experiment Setup, Data Preprocessing and Performance Evaluation Metrics

Our research involves multiple models and multiple datasets, which demand distinct preprocessing for each case. In the case of the ROAD dataset, the logs are not labeled, so we parsed the metadata



**Figure 3: Sequence creation process for the TAN.**

made available by the authors and used it to systematically parse and label each sample. Finally, we merged every log for the multiclass classification task, while we separately merged each log of every attack type for binary classification, resembling the HCRL Car-Hacking dataset setup.

Commonly to every dataset, we parsed each field into CAN ID, DLC (unavailable for the ROAD dataset) and data field. We padded the latter to 8 bytes and then we divided it into 8 different features, allowing detection of attacks targeting specific bytes of this field. Each field is then converted from hexadecimal to decimal format, in order to be processed by the selected models, and every dataset is split into training and test sets with a 80/20 ratio.

It's worth noting that the ROAD dataset is highly imbalanced, as it can be seen on Table 1, and some classes need to be oversampled. For this reason we employed the SMOTE algorithm [4] on the training set to increase the number of samples of each attack to a fixed value of 100,000, therefore ensuring an equal distribution between them, that otherwise could have not been detected from any metric produced by our models after building the sequences. The In-Vehicle Network Intrusion Detection Challenge dataset has been oversampled in the same way. The resulting data can be directly fed to the traditional machine learning models and the LSTM-based model, but it needs to undergo further processing in order to create the frames needed by the DCNN and the sequences needed by the TAN, built as described in Section 4, in Figure 2 and Figure 3.

The Random Forest, LightGBM, LCCDE and LSTM-based model perform multiclass classification on the full ROAD dataset, merged and oversampled. This creates the results with the highest fidelity to the real-world scenario. On the contrary, the TAN, the DCNN and the improved DCNN version, together with the binary version of the LSTM-based model, perform binary classification for every different attack type, and they are trained and tested on each attack type's data which has been separately merged to resemble the setup of the HCRL Car-Hacking dataset and In-Vehicle Network Intrusion Detection Challenge dataset, in order to compare the results, so they eventually produce a different model per attack class.

Since the choice of performance evaluation metrics is crucial in well representing a model's effectiveness for intrusion detection systems, we have opted for precision, recall and the F1 score, in order to correctly detect false positives and false negatives, while preserving a general value useful to evaluate the overall performance and therefore compare the models.

The experimental environment in which every model has been tested is on a system with a NVIDIA A100 GPU with 40GB of VRAM, 387.57GB of RAM, and two AMD EPYC 7302 16-Core Processors. The libraries to recreate the machine learning and deep learning models include PyTorch, TensorFlow and scikit-learn.

## 5.2 Experimental Results and Discussion of the Models

**5.2.1 LCCDE and ML approaches.** The performance of the LCCDE model, consisting of three gradient boosting classifiers (LightGBM, CatBoost and XGBoost), has been reproduced on the HCRL Car-Hacking dataset and we obtained very good and constant results on the In-Vehicle Network Intrusion Detection Challenge dataset. However, the model has not performed as well on the ROAD dataset, despite having analyzed both the ID and data field, with an average F1 score of 0.3212 and standard deviation of 0.1723. It is worth noting that this model could not detect at all two of the five masquerade attacks. A similar situation involves Random Forests which on average outperform slightly LightGBM, but had worse results than LCCDE. Furthermore, the LCCDE model was the one that required the most computation time due to the original implementation and impossibility of parallelization, requiring about 32 hours and 40 minutes of running time for the classification task on the ROAD dataset, while LightGBM has only taken about 6 minutes, XGBoost has taken about 10 minutes and CatBoost has taken about one hour and 27 minutes, which means that the vast majority of the computation time is due to the LCCDE algorithm. The code for the LCCDE model was made available by the authors [13].

Despite being more explainable and lightweight in terms of memory usage, these models were not able to reach satisfactory results

on the ROAD dataset, therefore we can conclude that traditional machine learning approaches are not fit for this task.

**5.2.2 LSTM-based model.** The model showed an exceptional performance for binary classification on the ROAD dataset (Table 7), perfectly classifying almost every attack type except the fuzzing attack, where it still reached a very good F1 score of 0.9977. Unfortunately, a binary classification approach is not ideal for the purpose of an IDS, as it would require a different pre-trained model for every different attack type, making it particularly expensive and challenging to update. We can conclude that its real-world application is questionable.

For the multiclass classification on the ROAD dataset, the LSTM-based model was able to achieve, on average, the best performance compared to the other traditional machine learning models. Nonetheless, its performance is largely unsatisfying and the perfect recall and low precision for most of the attack types show a tendency to produce a large number of false positives. The model was able to obtain a good F1 score on the fuzzing attack thanks to the simplicity in detecting the data field set to the maximum value (0xFFFFFFFFFFFF) for every attack message. As for the other models, it reached particularly negative results on the Max Engine Coolant Temp Attacks for which it was unable to correctly classify any attack sample in the test set, likely due to their insufficient number (Table 1).

The opposite goes for the model's results on the Car-Hacking dataset and on the In-Vehicle Network Intrusion Detection Challenge dataset, where its average F1 score is 0.9778, while on the ROAD dataset is 0.5963, highlighting a great performance divergence between detecting simpler attacks and more realistic ones. However, there is a noticeable difference between its performance on the fuzzing attack of the ROAD dataset, where it reached an average F1 score of 0.9975, and on the fuzzy attack of the HCRL Car Hacking dataset, where the average F1 score was 0.9179. This difference is due to the nature of the attacks, even if they have been named similarly: the former has a clear attack pattern, while the latter injects messages with completely random CAN ID and DATA values.

**5.2.3 DCNN.** We have been able to reproduce the performance shown on the HCRL Car-Hacking dataset, which was built by the same authors of this model [24]. The results were not as stable on the In-Vehicle Network Intrusion Detection Challenge dataset, having a higher average standard deviation of 0.05 on the F1 score metric, while the model has seen worse performance and dramatic fluctuations on the ROAD dataset. This has probably been caused by the fact that this dataset contains attacks acting also on the data field and they are much more difficult to detect because of their stealthiness, described on the Section 3.1. Considering this, the DCNN has performed fairly well, sometimes producing less false positives than the TAN model, with a notable exception for the fuzzing attack. This attack is easily identifiable by the payload, always set on the maximum value, but this model only uses the ID field, that in this case has cycling IDs. The cycling pattern is hardly detectable from the input frame containing 29 consecutive IDs, which explains the unreliability of the model.

As mentioned in Subsection 4.3, the original frame generation of the DCNN is using a sliding window of size 29 with a step of 29,

**Table 4: Model performance comparison for each class in the HCRL Car Hacking dataset.**

| Attacks              | Model   | Precision     | Recall        | F1 score      |
|----------------------|---------|---------------|---------------|---------------|
| DoS Attack           | RF      | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | LighGBM | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | LCCDE   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | DCNN    | <b>1.0000</b> | 0.9989        | 0.9995        |
|                      | TAN     | <b>1.0000</b> | 0.9961        | 0.9980        |
|                      | LSTM    | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| Fuzzy Attack         | RF      | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | LighGBM | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | LCCDE   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | DCNN    | 0.9995        | 0.9965        | 0.9980        |
|                      | TAN     | <b>1.0000</b> | 0.9999        | 0.9999        |
|                      | LSTM    | 0.9590        | 0.8802        | 0.9179        |
| Gear Spoofing Attack | RF      | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | LighGBM | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | LCCDE   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | DCNN    | 0.9999        | 0.9989        | 0.9994        |
|                      | TAN     | 0.9989        | 0.9926        | 0.9957        |
|                      | LSTM    | 0.9592        | <b>1.0000</b> | 0.9792        |
| RPM Spoofing Attack  | RF      | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | LighGBM | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | LCCDE   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                      | DCNN    | 0.9999        | 0.9994        | 0.9996        |
|                      | TAN     | 0.9996        | 0.9967        | 0.9982        |
|                      | LSTM    | 0.9998        | <b>1.0000</b> | <b>1.0000</b> |

which means that some attack patterns shown in ID bits could be cut into two different frames. For this reason we created a different version of frame generation, employing a step equal to one, and we tested it on the ROAD dataset, labeling it as DCNNv2. As Table 7 shows, thanks to this small modification, the model could reliably and constantly improve its F1 score, going from an average of 0.6514 to an average of 0.7468, and it was able to beat the F1 score of the TAN for the majority of the attack types. Nonetheless, as for the LSTM-based model and the TAN, this approach is hardly applicable in a real-world scenario, as it would require multiple models running in parallel to detect only predefined attacks, other than a non negligible overhead for the frame generation.

**5.2.4 TAN.** The TAN was able to maintain a good performance for most of the attacks, and it reached very good F1 scores for most of the attack types except for the Max Engine Coolant Temp Attacks, which, as mentioned earlier, only have few attack samples available. Furthermore, its recall is always higher than its precision (as Table 7 shows), even drastically in the case of the variations of the Max Engine Coolant Temp Attacks, which means that it produces more false positives than false negatives.

Like we previously mentioned, this model performs binary classification on sequences of CAN IDs, so it cannot distinguish the attack class detected and it was tested on every attack type singularly.

**Table 5: Performance evaluation of models on the In-Vehicle Network intrusion detection Challenge dataset.**

| Vehicle         | Model    | Precision       | Recall          | F1 score        |
|-----------------|----------|-----------------|-----------------|-----------------|
| Chevrolet Spark | RF       | 99.9984         | <b>99.9984</b>  | <b>99.9984</b>  |
|                 | LightGBM | 99.9952         | 99.9952         | 99.9952         |
|                 | LCCDE    | 99.9991         | 99.9167         | 99.9578         |
|                 | DCNN     | 85.5856         | 97.9603         | 90.1321         |
|                 | TAN      | <b>100.0000</b> | 99.7443         | 99.8719         |
|                 | LSTM     | 98.5301         | 99.9083         | 99.2144         |
| Hyundai Sonata  | RF       | <b>100.0000</b> | <b>100.0000</b> | <b>100.0000</b> |
|                 | LightGBM | 99.9936         | 99.9936         | 99.9936         |
|                 | LCCDE    | <b>100.0000</b> | <b>100.0000</b> | <b>100.0000</b> |
|                 | DCNN     | <b>100.0000</b> | 99.1681         | 99.5816         |
|                 | TAN      | 99.9287         | 99.4970         | 99.8010         |
|                 | LSTM     | <b>100.0000</b> | 99.9265         | 99.9632         |
| Kia Soul        | RF       | <b>99.9974</b>  | <b>99.9974</b>  | <b>99.9974</b>  |
|                 | LightGBM | 99.9965         | 99.9965         | 99.9965         |
|                 | LCCDE    | 99.9226         | 99.9828         | 99.9526         |
|                 | DCNN     | 99.3859         | 99.7984         | 99.5909         |
|                 | TAN      | 99.7438         | 99.6071         | 99.6754         |
|                 | LSTM     | 99.9877         | 95.8677         | 97.8844         |

## 6 Conclusion

After an accurate analysis of different models on the novel ROAD dataset and two datasets established in previous literature, the HCRL Car-Hacking dataset and the In-Vehicle Network Intrusion Detection Challenge dataset, we encountered a sensible disparity in the results. Every model performing multiclass classification that had shown almost perfect predictions on the former datasets, demonstrated to be highly ineffective on the latter, rarely reaching satisfactory results. On the contrary, the models performing binary classification had kept good precision, recall and F1 score for most of the attack types. However, this is not due to the intrinsic difficulty of multiclass classification of multivariate data, but to the setup in which the latter have been usually tested, which involves binary classification performed on different datasets, each containing one isolated attack class. This case is scarcely indicative of a real-world scenario because, to detect each class, it would require multiple models being deployed at the same time, each trained on one predefined attack, therefore increasing the resources required as well as compromising their ability of detecting new attack types as well. Thus, this setup produces exceptionally good metrics that unfortunately do not translate well with the models' practical performance. Furthermore, the authors of these models have not taken into consideration their complexity in terms of memory and processing time, which would be a fundamental aspect of in-vehicle detection, due to the limited computational resources available.

Given the stealthier attacks of the ROAD dataset which better represent the real-world case, this dataset should be considered as more fit to represent a realistic scenario, therefore it should be included in the test benches on which new IDSs would be benchmarked, and new datasets should be built aiming in this direction.

However, as shown by the number of samples on Table 1, the ROAD dataset is not without flaw, presenting a number of samples too small for deep learning approaches and a heavy class imbalance,

**Table 6: Model performance comparison for multiclass classification on the ROAD data.**

| Attacks         | Model    | Precision     | Recall        | F1 score      |
|-----------------|----------|---------------|---------------|---------------|
| Correlated      | RF       | <b>0.5068</b> | <b>1.0000</b> | 0.6727        |
| Signal          | LightGBM | 0.0000        | 0.0000        | 0.0000        |
| Fabrication     | LCCDE    | 0.5014        | <b>1.0000</b> | <b>0.6766</b> |
| Attack          | LSTM     | 0.5018        | <b>1.0000</b> | 0.6683        |
| Correlated      | RF       | 0.0000        | 0.0000        | 0.0000        |
| Signal          | LightGBM | <b>0.2725</b> | <b>1.0000</b> | <b>0.4213</b> |
| Masquerade      | LCCDE    | 0.0000        | 0.0000        | 0.0000        |
| Attack          | LSTM     | 0.0000        | 0.0000        | 0.0000        |
| Fuzzing Attack  | RF       | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
|                 | LightGBM | 0.0134        | 0.6236        | 0.0311        |
|                 | LCCDE    | 0.2674        | <b>1.0000</b> | 0.4224        |
|                 | LSTM     | 0.9951        | <b>1.0000</b> | 0.9975        |
| Max Engine      | RF       | <b>0.6667</b> | <b>1.0000</b> | <b>0.8000</b> |
| Coolant Temp    | LightGBM | 0.0000        | 0.0000        | 0.0000        |
| Fabrication     | LCCDE    | 0.2500        | <b>1.0000</b> | 0.4000        |
| Attack          | LSTM     | 0.0000        | 0.0000        | 0.0000        |
| Max Engine      | RF       | 0.0000        | 0.0000        | 0.0000        |
| Coolant Temp    | LightGBM | 0.0000        | 0.0000        | 0.0000        |
| Masquerade      | LCCDE    | 0.0000        | 0.0000        | 0.0000        |
| Attack          | LSTM     | <b>0.7500</b> | <b>1.0000</b> | <b>0.8571</b> |
| Max             | RF       | 0.1257        | 0.1204        | 0.1229        |
| Speedometer     | LightGBM | 0.2944        | <b>0.4512</b> | <b>0.3536</b> |
| Fabrication     | LCCDE    | 0.3713        | 0.2384        | 0.2822        |
| Attack          | LSTM     | <b>0.4335</b> | 0.0586        | 0.1032        |
| Max             | RF       | 0.1408        | 0.1469        | 0.1438        |
| Speedometer     | LightGBM | 0.1715        | 0.1647        | 0.1693        |
| Masquerade      | LCCDE    | 0.3612        | 0.3612        | 0.3612        |
| Attack          | LSTM     | <b>0.4976</b> | <b>0.9241</b> | <b>0.6469</b> |
| Reverse Light   | RF       | 0.2514        | 0.2421        | 0.2467        |
| Off Fabrication | LightGBM | 0.0646        | 0.3173        | 0.1125        |
| Attack          | LCCDE    | <b>0.3655</b> | 0.3836        | <b>0.3712</b> |
|                 | LSTM     | 0.2432        | <b>0.4684</b> | 0.3201        |
| Reverse Light   | RF       | 0.2719        | 0.2846        | 0.2781        |
| Off             | LightGBM | 0.2511        | 0.3453        | 0.2946        |
| Masquerade      | LCCDE    | <b>0.3612</b> | 0.3334        | 0.3454        |
| Attack          | LSTM     | 0.2964        | <b>0.4888</b> | <b>0.3690</b> |
| Reverse Light   | RF       | 0.1567        | 0.1532        | 0.1550        |
| On Fabrication  | LightGBM | 0.2302        | 0.4988        | 0.3133        |
| Attack          | LCCDE    | 0.3221        | 0.3423        | 0.3384        |
|                 | LSTM     | <b>0.4310</b> | <b>0.8239</b> | <b>0.5659</b> |
| Reverse Light   | RF       | 0.1702        | 0.1741        | 0.1721        |
| On              | LightGBM | 0.0741        | 0.1889        | 0.1010        |
| Masquerade      | LCCDE    | 0.3222        | <b>0.2993</b> | <b>0.3144</b> |
| Attack          | LSTM     | <b>0.3951</b> | 0.1404        | 0.2071        |

particularly for the Max Engine Coolant Temp Fabrication Attack and the corresponding masquerade attack. Indeed, every model produced their worst results on these two classes, supporting the hypothesis that the number of attack samples is insufficient for this task.

**Table 7: Model performance comparison for binary classification on the ROAD data.**

| Attacks           | Model  | Precision     | Recall        | F1 score      |
|-------------------|--------|---------------|---------------|---------------|
| Correlated Signal | DCNN   | 0.9919        | 0.9388        | 0.9644        |
| Fabrication       | DCNNv2 | 0.9981        | 0.9889        | 0.9934        |
| Attack            | TAN    | 0.9504        | <b>1.0000</b> | 0.9746        |
|                   | LSTM   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| Correlated Signal | DCNN   | 0.9947        | 0.6999        | 0.8130        |
| Masquerade        | DCNNv2 | 0.9997        | 0.9194        | 0.9578        |
| Attack            | TAN    | 0.9437        | <b>1.0000</b> | 0.9713        |
|                   | LSTM   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| Fuzzing Attack    | DCNN   | 0.2939        | 0.3104        | 0.2430        |
|                   | DCNNv2 | 0.2929        | 0.1264        | 0.1764        |
|                   | TAN    | <b>0.9960</b> | 0.9994        | 0.9976        |
|                   | LSTM   | 0.9955        | <b>1.0000</b> | <b>0.9977</b> |
| Max Engine        | DCNN   | 0.0836        | 0.6571        | 0.1323        |
| Coolant Temp      | DCNNv2 | 0.8163        | 0.0744        | 0.1230        |
| Fabrication       | TAN    | 0.5334        | <b>1.0000</b> | 0.6957        |
| Attack            | LSTM   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| Max Engine        | DCNN   | 0.1161        | 0.4381        | 0.1369        |
| Coolant Temp      | DCNNv2 | 0.6510        | 0.1044        | 0.1700        |
| Masquerade        | TAN    | 0.0098        | 0.9535        | 0.0194        |
| Attack            | LSTM   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| Max               | DCNN   | 0.9457        | 0.5039        | 0.6511        |
| Speedometer       | DCNNv2 | 0.9617        | 0.8543        | 0.9049        |
| Fabrication       | TAN    | 0.9884        | <b>1.0000</b> | 0.9941        |
| Attack            | LSTM   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| Max               | DCNN   | 0.9893        | 0.4629        | 0.6058        |
| Speedometer       | DCNNv2 | 0.9998        | 0.8880        | 0.9404        |
| Masquerade        | TAN    | 0.9881        | 0.9986        | 0.9933        |
| Attack            | LSTM   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| Reverse Light     | DCNN   | 0.7945        | 0.9733        | 0.8710        |
| Off Fabrication   | DCNNv2 | 0.9533        | 0.9992        | 0.9756        |
| Attack            | TAN    | 0.9753        | 0.9884        | 0.9818        |
|                   | LSTM   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| Reverse Light     | DCNN   | 0.9852        | 0.8000        | 0.8780        |
| Off               | DCNNv2 | 0.9981        | 0.9953        | 0.9967        |
| Masquerade        | TAN    | 0.9641        | 0.9883        | 0.9761        |
| Attack            | LSTM   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| Reverse Light     | DCNN   | 0.8871        | 0.9644        | 0.9232        |
| On Fabrication    | DCNNv2 | 0.9600        | 0.9951        | 0.9772        |
| Attack            | TAN    | 0.9600        | 0.9743        | 0.9670        |
|                   | LSTM   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| Reverse Light     | DCNN   | 0.9993        | 0.9069        | 0.9467        |
| On                | DCNNv2 | 0.9998        | 0.9990        | 0.9994        |
| Masquerade        | TAN    | 0.9593        | 0.9744        | 0.9668        |
| Attack            | LSTM   | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |

Our research has not focused on the reaction of these models in the presence of unseen attacks, which are expected to degrade their overall performance, particularly in traditional machine learning models like Random Forests. To provide accurate metrics for IDS evaluation, we believe that researchers should challenge their models with new attacks that are completely excluded from the training set.

Finally, although we have not examined the explainability of these models, we believe that the interpretability of the predictions and the ability to define cause-and-effect relationships could provide relevant insights, such as explaining the nature and root cause of a single intrusion. This would allow for more accurate intrusion prevention and the adaptation of IDSs to a wider range of threats, making them more generalizable and effective.

## Acknowledgments

This paper is supported by the European Union's HORIZON Research and Innovation Programme under grant agreement No 101120657, project ENFIELD (European Lighthouse to Manifest Trustworthy and Green AI). The authors also appreciate the support from the University of Bologna which granted Lorenzo Guerra a scholarship for the preparation of his Master's thesis abroad.

## References

- [1] Bifta Sama Bari, Kumar Yelamarthi, and Sheikh Ghafoor. 2023. Intrusion Detection in Vehicle Controller Area Network (CAN) Bus Using Machine Learning: A Comparative Performance Study. *Sensors* 23, 7 (2023). <https://doi.org/10.3390/s23073610>
- [2] Rohit Bhatia, Vireshwar Kumar, Khaled Serag, Z. Berkay Celik, Mathias Payer, and Dongyan Xu. 2021. Evading Voltage-Based Intrusion Detection on Automotive CAN. *Proceedings 2021 Network and Distributed System Security Symposium* (2021). <https://api.semanticscholar.org/CorpusID:222008310>
- [3] Linux CAN. 2015. *Linux-CAN SocketCAN user space applications*. <https://github.com/linux-can/can-utils> Accessed: 2024-07-03.
- [4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. 2002. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research* 16 (2002), 321–357. <https://doi.org/10.1613/jair.953>
- [5] Stephen Checkoway, Damon McCoy, Brian Kantor, Danny Anderson, Hovav Shacham, Stefan Savage, Karl Koscher, Alexei Czeskis, Franziska Roesner, and Tadayoshi Kohno. 2011. Comprehensive experimental analyses of automotive attack surfaces. In *Proceedings of the 20th USENIX Conference on Security* (San Francisco, CA) (SEC'11). USENIX Association, USA, 6.
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. [arXiv:2010.11929](https://arxiv.org/abs/2010.11929) [cs.CV]
- [7] Hacking and Countermeasure Research Lab. 2018. *HCRL Car-Hacking Dataset*. <https://web.archive.org/web/20240703084210/https://ocslab.hksecurity.net/Datasets/car-hacking-dataset> Accessed: 2024-07-03.
- [8] Hacking and Countermeasure Research Lab. 2018. *HCRL Survival Analysis Dataset*. <https://web.archive.org/web/20240703084625/https://ocslab.hksecurity.net/Datasets/survival-ids> Accessed: 2024-07-03.
- [9] Hacking and Countermeasure Research Lab. 2019. *In-Vehicle Network Intrusion Detection Challenge 2019 dataset*. <https://web.archive.org/web/20231203064441/https://ocslab.hksecurity.net/Datasets/datachallenge2019/car> Accessed: 2024-07-03.
- [10] Armin Happel. 2014. *Secure communication for CAN FD*. [https://web.archive.org/web/20221007133906/https://cdn.vectron.com/cms/content/know-how/\\_technical-articles/Security\\_CAN\\_Newsletter\\_201411\\_PressArticle\\_EN.pdf](https://web.archive.org/web/20221007133906/https://cdn.vectron.com/cms/content/know-how/_technical-articles/Security_CAN_Newsletter_201411_PressArticle_EN.pdf) Accessed: 2024-07-04.
- [11] Md Delwar Hossain, Hiroyuki Inoue, Hideya Ochiai, Doudou Fall, and Youki Kadobayashi. 2020. LSTM-Based Intrusion Detection System for In-Vehicle Can Bus Communications. *IEEE Access* 8 (2020), 185489–185502. <https://doi.org/10.1109/ACCESS.2020.3029307>
- [12] Karl Koscher, Alexei Czeskis, Franziska Roesner, Shwetak Patel, Tadayoshi Kohno, Stephen Checkoway, Damon McCoy, Brian Kantor, Danny Anderson, Hovav Shacham, and Stefan Savage. 2010. Experimental Security Analysis of a Modern Automobile. In *2010 IEEE Symposium on Security and Privacy*. 447–462. <https://doi.org/10.1109/SP.2010.34>
- [13] Western OC2 Lab. 2021. *Code for IDS-ML: intrusion detection system development using machine learning algorithms*. <https://github.com/Western-OC2-Lab/Intrusion-Detection-System-Using-Machine-Learning/> Accessed: 2024-07-03.
- [14] Brooke Lampe and Weizhi Meng. 2023. *can-train-and-test: A Curated CAN Dataset for Automotive Intrusion Detection*. [arXiv:2308.04972](https://arxiv.org/abs/2308.04972) [cs.CR]
- [15] Hsiao-Chung Lin, Ping Wang, Kuo-Ming Chao, Wen-Hui Lin, and Jia-Hong Chen. 2022. Using Deep Learning Networks to Identify Cyber Attacks on Intrusion

- Detection for In-Vehicle Networks. *Electronics* 11, 14 (2022). <https://doi.org/10.3390/electronics11142180>
- [16] Siti-Farhana Lokman, Abu Talib Othman, and Muhammad-Husaini Abu-Bakar. 2019. Intrusion detection system for automotive Controller Area Network (CAN) bus system: a review. *EURASIP Journal on Wireless Communications and Networking* 2019 (2019), 1–17. <https://doi.org/10.1186/s13638-019-1484-3>
- [17] Miller and Valasek. 2014. A Survey of Remote Automotive Attack Surfaces. <https://web.archive.org/web/20240703092026/https://illmatics.com/remot%20attack%20surfaces.pdf> Accessed: 2024-07-03.
- [18] Miller and Valasek. 2015. Remote Exploitation of an Unaltered Passenger Vehicle. <https://web.archive.org/web/20240703220542/https://illmatics.com/Remote%20Car%20Hacking.pdf> Accessed: 2024-07-03.
- [19] Miller and Valasek. 2016. CAN Message Injection. <https://web.archive.org/web/20240303060608/https://illmatics.com/can%20message%20injection.pdf> Accessed: 2024-07-03.
- [20] BBC News. 2015. *Fiat Chrysler recalls 1.4 million cars after Jeep hack*. <https://web.archive.org/web/20240509102340/https://www.bbc.com/news/technology-33650491> Accessed: 2024-07-03.
- [21] Trieu Phong Nguyen, Heungwoo Nam, and Daehee Kim. 2023. Transformer-Based Attention Network for In-Vehicle Intrusion Detection. *IEEE Access* 11 (2023), 55389–55403. <https://doi.org/10.1109/ACCESS.2023.3282110>
- [22] Sampath Rajapaksha, Harsha Kalutarage, M. Omar Al-Kadri, Andrei Petrovski, Garikayi Madzudzo, and Madeline Cheah. 2023. AI-Based Intrusion Detection Systems for In-Vehicle Networks: A Survey. *ACM Comput. Surv.* 55, 11, Article 237 (2023), 40 pages. <https://doi.org/10.1145/3570954>
- [23] Bosch Semiconductor. 2017. *CAN Specification Verion 2.0*. [https://web.archive.org/web/20170621031900/http://www.bosch-semiconductors.de/media/ubk\\_semiconductors/pdf\\_1/canliteratur/can2spec.pdf](https://web.archive.org/web/20170621031900/http://www.bosch-semiconductors.de/media/ubk_semiconductors/pdf_1/canliteratur/can2spec.pdf) Accessed: 2024-07-03.
- [24] Hyun Min Song, Jiyoung Woo, and Huy Kang Kim. 2020. In-vehicle network intrusion detection using deep convolutional neural network. *Vehicular Communications* 21 (2020), 100198. <https://doi.org/10.1016/j.vehcom.2019.100198>
- [25] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alex Alemi. 2016. Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. arXiv:1602.07261 [cs.CV] <https://arxiv.org/abs/1602.07261>
- [26] Chris Valasek and Charlie Miller. 2014. Adventures in Automotive Networks and Control Units. [https://web.archive.org/web/20240704143111/https://ioactive.com/pdfs/IOActive\\_Adventures\\_in\\_Automotive\\_Networks\\_and\\_Control\\_Units.pdf](https://web.archive.org/web/20240704143111/https://ioactive.com/pdfs/IOActive_Adventures_in_Automotive_Networks_and_Control_Units.pdf)
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762
- [28] Miki E. Verma, Robert A. Bridges, Michael D. Iannacone, Samuel C. Hollifield, Pablo Moriano, Steven C. Hespeler, Bill Kay, and Frank L. Combs. 2024. A comprehensive guide to CAN IDS data and introduction of the ROAD dataset. *PLOS ONE* 19, 1 (01 2024), 1–32. <https://doi.org/10.1371/journal.pone.0296879> <https://zenodo.org/records/10462796>
- [29] Li Yang, Abdallah Shami, Gary Stevens, and Stephen de Rusett. 2022. LCCDE: A Decision-Based Ensemble Framework for Intrusion Detection in The Internet of Vehicles. In *GLOBECOM 2022 - 2022 IEEE Global Communications Conference*. 3545–3550. <https://doi.org/10.1109/GLOBECOM48099.2022.10001280>